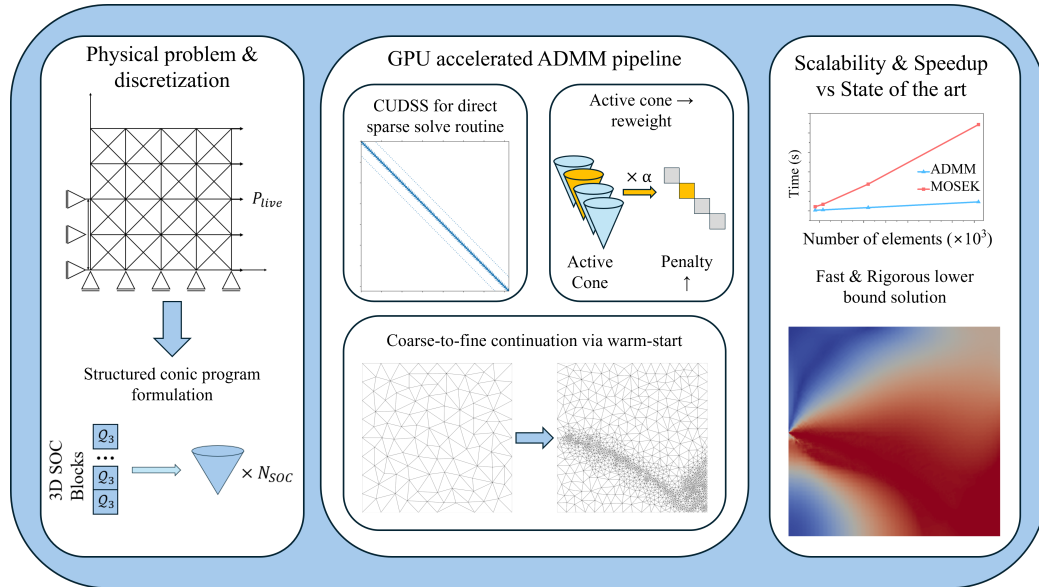


Graphical Abstract

Efficient GPU-Accelerated ADMM for Large-Scale Lower-Bound Finite Element Limit Analysis

Yunfei Gao, Chris Martin, Paul Goulart



Highlights

Efficient GPU-Accelerated ADMM for Large-Scale Lower-Bound Finite Element Limit Analysis

Yunfei Gao, Chris Martin, Paul Goulart

- GPU-accelerated ADMM pipeline for million-variable LB-FELA SOCPs, outperforming IPM baselines in time-to-solution while preserving accuracy.
- Active-cone identification and dynamic penalty reweighting for SOC blocks to accelerate convergence in block-structured SOCPs.
- Coarse-to-fine continuation with warm starts across mesh resolutions, substantially reducing iteration counts for the same benchmark under finer mesh discretizations.
- GPU linear-solve and projection kernels with symbolic factorization reuse, enabling efficient large-scale computations.

Efficient GPU-Accelerated ADMM for Large-Scale Lower-Bound Finite Element Limit Analysis

Yunfei Gao, Chris Martin, Paul Goulart

Department of Engineering Science, University of Oxford, Parks Road, Oxford, OX1 3PJ, Oxfordshire, UK

Abstract

We propose a GPU-accelerated alternating direction method of multipliers (ADMM) framework for solving large-scale lower-bound (LB) finite element limit analysis (FELA) problems. The method incorporates active-cone identification with dynamic penalty reweighting, as well as a coarse-to-fine continuation strategy that exploits ADMM’s warm-start capability to improve convergence on large, sparse conic optimization problems. The dominant sparse linear solves and vector operations are implemented on the GPU using CUDA, significantly reducing per-iteration cost. The proposed framework is systematically evaluated on large-scale 2D and 3D FELA benchmarks formulated as second-order cone programming (SOCP) problems, where it demonstrates substantial speedups over a commercial interior-point solver while maintaining comparable accuracy. In addition, we demonstrate that the same framework extends naturally to representative 3D FELA problems. These results indicate that GPU-accelerated ADMM provides an efficient and extensible approach for large-scale FELA, combining strong performance in 2D with practical applicability to 3D problems.

Keywords: Alternating direction of multipliers method, Finite element limit analysis, Second-order cone programming, GPU computing

1. Introduction

Interior-point methods (IPMs) have long been the standard approach for solving such conic optimization problems, owing to their robustness, high accuracy, and fast convergence in terms of iteration count. Over the past two decades, both general-purpose conic solvers and tailored FELA solvers have been developed around this methodology. General-purpose solvers such as CVXOPT, ECOS, MOSEK, and Clarabel provide mature, robust primal-dual interior-point implementations for SOCPs, while tailored FELA solvers further exploit problem-specific structures, including Schur-complement systems, variable elimination, matrix reordering, and parallel factorization [1, 2, 3, 4, 5, 6]. These solvers vary in their presolve techniques, problem formulations, and linear-solve strategies, yet they commonly rely on repeatedly forming and factorizing iteration-dependent KKT systems. As the problem size increases, particularly in 3D finite element analyses, the resulting factorization cost and fill-in can lead to substantial memory demands, making sparse direct methods difficult to apply on very large instances [7]. Indirect or iterative linear solvers are therefore an attractive alternative for reducing memory consumption [8]. However, the linear systems arising in IPMs often become increasingly ill-conditioned as the iterates approach convergence, which can significantly degrade the effectiveness of indirect methods [9], though that problem may be alleviated by the use of carefully designed preconditioners [10, 11].

Recent advances have seen the application of a first-order optimization method, ADMM to FELA [12, 13], demonstrating its potential as a robust solver that can yield answers with good accuracy in a reasonable amount of time. Compared with IPMs, ADMM offers a different computational trade-off: it typically requires more iterations, but each iteration can be significantly cheaper. It uses only first-order derivative information, bypassing the need to reformulate the Hessian matrix every iteration. Therefore ADMM is able to store and reuse the results obtained from the initial numerical factorization, thereby benefiting from cheaper per-iteration computational cost and low memory requirement.

Another attractive feature of ADMM in FELA is its projection-centric treatment of yield constraints. Previous studies have reported successful applications of ADMM to more sophisticated 3D yield criteria, such as Lade-Duncan and Hoek-Brown criteria [14, 15]. This projection-centric treatment provides a practical alternative to interior-point formulations that require the

yield locus to admit an explicit conic (SOCP/SDP) representation, which can be restrictive for more sophisticated criteria. Although the present work focuses on SOCP formulations, this flexibility is important for future extensions to more general yield models that are more common in geotechnical practice.

Despite these advantages, ADMM usually requires more iterations than IPMs to converge to a high-accuracy solution, while IPMs often reach high accuracy in only a few tens of iterations [12, 16]. Moreover, the convergence rate of ADMM is strongly affected by the penalty parameters in the augmented Lagrangian. In OSQP, for example, the penalty parameter is chosen adaptively, aiming to balance the primal and dual residuals, with larger penalties assigned to constraints that are expected to be active [17]. In SuperADMM, constraint-wise penalty parameters are updated dynamically to accelerate convergence [18], although this has only been applied to quadratic programming with linear constraints. Motivated by these ideas, we seek to improve ADMM convergence for the large-scale SOCPs arising in lower-bound FELA through two strategies: coarse-to-fine warm-starting and active-cone based dynamic penalty reweighting.

Several general-purpose ADMM-based solvers have been developed for large-scale optimization. OSQP targets quadratic programming and supports warm-starting and adaptive penalty updates [17]. COSMO extends ADMM to conic optimization problems through quasi-definite linear systems and projections onto convex cones, while also supporting modular extensions such as approximate projections and GPU parallelism [19]. SCS applies ADMM to the homogeneous self-dual embedding of conic programs and supports both direct and matrix-free indirect linear-system solves [20]. Recent work has also explored approximate ADMM strategies in which the linear-system solve or cone projection is performed inexactly, for example through iterative eigensolvers for semidefinite projections or indirect linear solvers [21, 9].

Although general-purpose ADMM solvers such as OSQP, COSMO, and SCS have demonstrated the effectiveness of operator-splitting methods for large-scale quadratic programming (QP) and conic optimization problems [17, 19, 20], they are not specifically designed to exploit the structure of lower-bound FELA. The problems considered here consist of global sparse linear equilibrium constraints coupled with a large number of local second-order cone yield constraints, many of which share the same dimension and projection structure. Moreover, adaptive mesh refinement naturally generates a sequence of closely related SOCP instances that differ only in mesh discretization, for which it is natural to employ a warm-start method. Effi-

ciently exploiting this structure enables several problem-specific strategies, including highly parallelizable SOC projections, coarse-to-fine warm-starting, active-cone based penalty reweighting, and, in future work, carefully designed preconditioners for indirect linear-solver methods. These considerations motivate the development of a tailored GPU-accelerated ADMM solver for large-scale lower-bound FELA.

In this paper, we develop a tailored ADMM solver for large-scale SOCPs arising in 2D and 3D lower-bound FELA. The main contributions are:

1. We derive a tailored ADMM formulation for the resulting SOCPs, including the condensed linear system, cone projection steps, termination criteria, and two optional enhancements: ellipsoidal projection and an augmented linear-system formulation.
2. We propose two acceleration strategies for large-scale adaptive FELA: a coarse-to-fine warm-start strategy and an active-cone based dynamic penalty reweighting scheme inspired by SuperADMM [18].
3. We implement the dominant computational kernels on the GPU, including sparse direct solves using cuDSS [22], customized vector operations and cone projections, and cuBLAS/cuSPARSE-based matrix operations, thereby minimizing CPU-GPU data transfer.
4. We evaluate the proposed framework on representative 2D and 3D lower-bound FELA benchmarks, including rigid platen, strip footing, and vertical cut problems, and discuss the distinct computational bottlenecks and speedup mechanisms in two and three dimensions.

2. Implementation of an ADMM solver

2.1. ADMM Formulation and Updates

We consider the following standard-form second-order cone program (SOCP), consisting of a linear objective function, affine equality constraints, and conic constraints:

$$\begin{aligned} & \textbf{minimize} && c^\top x \\ & \textbf{subject to} && Ax + s = b, \\ & && s \in \mathcal{K}. \end{aligned} \tag{1}$$

Here, $x \in \mathbb{R}^n$ is the decision variable, $c \in \mathbb{R}^n$ defines the linear objective function, $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ define the affine coupling constraints, and $s \in \mathbb{R}^m$ is a slack variable introduced to express the constraints in conic form.

The cone $\mathcal{K} \subseteq \mathbb{R}^m$ is defined as the Cartesian product of a zero cone and a collection of second-order cones:

$$\mathcal{K} = \{0\}^{N_l} \times \prod_{i=1}^{N_{\text{SOC}}} \mathcal{Q}^{q_i}, \tag{2}$$

where N_l denotes the number of affine equality constraints, N_{SOC} denotes the number of second-order-cone blocks, and q_i denotes the dimension of the i -th cone block. Accordingly, the total number of constraints is

$$m = N_l + \sum_{i=1}^{N_{\text{SOC}}} q_i. \tag{3}$$

The zero-cone component $\{0\}^{N_l}$ fixes the corresponding entries of s to zero, thereby enforcing those constraints as equalities. The q_i -dimensional second-order cone is defined as

$$\mathcal{Q}^{q_i} = \{(t, \mathbf{r}) \in \mathbb{R} \times \mathbb{R}^{q_i-1} \mid \|\mathbf{r}\|_2 \leq t\}. \tag{4}$$

The detailed construction of A , b , and the individual cone blocks is presented in the following section. In the ADMM formulation, the variable splitting between x and s separates the affine and conic components of the problem. At each iteration, the x -subproblem associated with the affine coupling constraint is solved first, after which the slack variable is updated by projection onto $\mathcal{K}$. Owing to the Cartesian-product structure of $\mathcal{K}$, this projection is separable: the zero-cone components are set to zero, while each

remaining block is projected independently onto its corresponding second-order cone $\mathcal{Q}^{q_i}$.

Following the notation given in (1), we may solve this problem by ADMM by first giving the augmented Lagrangian as:

$$L_\rho(x, s, y) := c^\top x + \delta_{\mathcal{K}}(s) + y^\top (Ax + s - b) + \frac{\rho}{2} \|Ax + s - b\|^2 \quad (5)$$

where ρ is a parameter that controls the penalty weight imposed on the constraint $Ax + s = b$, and the indicator function $\delta_{\mathcal{K}}(s)$ refers to the condition $s \in \mathcal{K}$.

We may now specify our ADMM steps as follows:

1. **Primal variable update:** Minimize the augmented Lagrangian $L_\rho(x, s, y)$ with respect to x .
2. **Projection step:** Update s by projecting onto the intersecting cones, $\mathcal{K}$.
3. **Dual variable update:** Update the multiplier y .

2.1.1. Primal variable update

As depicted above, the first step of ADMM includes the minimization of the augmented Lagrangian. However, in an attempt to further improve the convergence behaviour of the proposed solver, we chose to adopt a non-uniform penalty parameter ρ . The operation of multiplying a scalar ρ corresponds to applying the same level of penalty across all constraints, whereas the proposed approach initializes the linear equality constraints with larger penalties. This distinction reflects the fact that these constraints are always active, whereas not all SOC constraints are necessarily active during the iterations. We also conduct regular checks on the active SOCs, and changing the corresponding penalty parameters in our implementation, the scheme will be elaborated in section 4.

Specifically, the first N_l linear equality constraints, that in the optimization process are always active, are assigned a large positive value, which is $10^4\rho$ as default, while the remaining entries are set to a small positive value, $\rho = 0.1$ as default, though the choice of this parameter largely depends on the specific optimization problem at hand. We implement the idea of assigning different weights to constraints as a multiplication of a diagonal matrix, $D(\rho)$, and its definition is as follows:

$$D(\rho) := \text{diag}(\rho_1, \dots, \rho_m), \quad \rho_i = \begin{cases} 10^4\rho & i \leq N_l \\ \rho & N_l < i \leq N \end{cases} \quad (6)$$

here N corresponds to the number of constraints in total.

The augmented Lagrangian now becomes:

$$L_\rho(x, s, y) := c^\top x + \delta_{\mathcal{K}}(s) + \frac{1}{2}(Ax + s - b + D(\rho)^{-1}y)^\top D(\rho)(Ax + s - b + D(\rho)^{-1}y) \quad (7)$$

taking its derivative with respect to x :

$$\Delta_x L_\rho(x, s, y) = c + A^\top D(\rho)Ax + A^\top D(\rho)(s - b + D(\rho)^{-1}y) = 0 \quad (8)$$

which equates to our final linear system:

$$A^\top D(\rho)Ax = -(c + A^\top D(\rho)(s - b + D(\rho)^{-1}y)) \quad (9)$$

The primal variable update may then be obtained by solving the linear system above. Notice that the left hand side matrix is a condensed matrix of shape $N_v \times N_v$, where N_v is the total number of variables. This matrix is sparse, symmetric and positive definite, which allows the use of Cholesky factorization routines.

2.1.2. Projection step

In principle, the slack variable is updated through the following formula:

$$s^{k+1} := \arg \min_s L_\rho(x^{k+1}, s, y^k) \quad (10)$$

Substituting the explicit form of the augmented Lagrangian, this becomes:

$$s^{k+1} := \arg \min_s \{ \delta_{\mathcal{K}}(s) + \frac{\rho}{2} \|s - (b - Ax^{k+1} - D(\rho)^{-1}y^k)\| \} \quad (11)$$

If we take out the identification function which says $s \in \mathcal{K}$, this further becomes:

$$s^{k+1} := \arg \min_{s \in \mathcal{K}} \|s - (b - Ax^{k+1} - D(\rho)^{-1}y^k)\| \quad (12)$$

This update corresponds to projecting the vector $v = b - Ax^{k+1} - D(\rho)^{-1}y^k$ onto the set of intersecting convex cones $\mathcal{K}$.

In our formulation, the first N_l elements of the slack variable vector corresponds to the linear equality constraints and should be always projected to zero. Then we are left with the slack variables for the yield criterion, satisfied at every stress point. The q_i -element segments of the i th SOC is specified as: $\mathbf{s}_i = (t_i, \mathbf{r}_i)$, where $i < N_{SOC}$, the total number of second-order

cones, the update of $\mathbf{s}_i$ of our q_i -dimension SOCs follows this closed-form expression [19]:

$$\mathbf{s}_i^{k+1} := \begin{cases} (0, 0, 0) & \text{if } \|\mathbf{r}_i\|_2 \leq -t_i \\ \left(\frac{t_i + \|\mathbf{r}_i\|_2}{2}, \frac{t_i + \|\mathbf{r}_i\|_2}{2\|\mathbf{r}_i\|_2} \cdot \mathbf{r}_i \right) & \text{if } \|\mathbf{r}_i\|_2 > t_i \\ \mathbf{s}_i^k & \text{otherwise} \end{cases} \quad (13)$$

2.1.3. Dual variable update

After the projection step, the dual variable y is then updated according to:

$$y^{k+1} := y^k + D(\rho)(Ax^{k+1} + s^{k+1} - b) \quad (14)$$

2.2. Termination criterion

Following the notation given in (1), we first obtain the expression of primal and dual residuals from the KKT conditions for convex optimization problems:

$$\begin{aligned} r_{\text{prim}} &= Ax + s - b \\ r_{\text{dual}} &= q + A^\top y \end{aligned} \quad (15)$$

We adopt the stopping criterion of [17], stating that the algorithm stops when the norms of the residuals r_{prim}^k and r_{dual}^k are smaller than some tolerance levels $\varepsilon_{\text{prim}} > 0$ and $\varepsilon_{\text{dual}} > 0$:

$$\|r_{\text{prim}}^k\|_\infty \leq \varepsilon_{\text{prim}} \quad \|r_{\text{dual}}^k\|_\infty \leq \varepsilon_{\text{dual}} \quad (16)$$

where $\varepsilon_{\text{prim}}$ and $\varepsilon_{\text{dual}}$ are obtained using prescribed absolute and relative tolerances, $\varepsilon_{\text{abs}} > 0$ and $\varepsilon_{\text{rel}} > 0$, using the formula below:

$$\varepsilon_{\text{prim}} = \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \max\{\|Ax^k\|_\infty, \|s\|_\infty, \|b\|_\infty\}, \quad (17)$$

$$\varepsilon_{\text{dual}} = \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \max\{\|A^\top y^k\|_\infty, \|c\|_\infty\}. \quad (18)$$

The default values for our solver are: $\varepsilon_{\text{abs}} = \varepsilon_{\text{rel}} = 1e - 4$

3. Lower-Bound FELA Formulation

In lower-bound FELA, the objective is to maximize the load multiplier β while constructing a statically admissible stress field. The stress field must satisfy equilibrium in each element, traction equilibrium across element interfaces, prescribed boundary tractions, and the yield criterion throughout the domain. After discretization, these requirements give rise to two classes of constraints: linear equality constraints associated with equilibrium and boundary conditions, and local conic constraints associated with yield admissibility.

For completeness, we briefly outline the LB-FELA formulation in both 2D and 3D, and the resulting conic constraints. A more detailed description of the LB-FELA discretization and the Bernstein-polynomial stress interpolation scheme may be found in [23, 24]. In this paper we present only the minimal formulation needed to formulate the SOCP structure and the proposed ADMM solver.

Element design plays a central role in maintaining the rigorous bound properties of FELA. In LB formulations, equilibrium and yield admissibility must be enforced in a way that guarantees their satisfaction throughout each element, while upper-bound formulations require the flow rule and kinematic boundary conditions to be satisfied throughout the discretized domain [25, 26]. Earlier rigorous FELA formulations commonly relied on constant-stress elements for LB analysis and linear or quadratic displacement elements for UB analysis. Higher-order UB formulations were later developed using six-noded linear strain triangular elements and SOCP-based discontinuous velocity formulations [27, 28]. More recently, Bernstein polynomial elements have provided a systematic alternative to classical Lagrange elements, allowing rigorous LB and UB formulations of arbitrary polynomial order for straight-sided triangular elements [24, 25, 26]. In this work, we adopt linear stress interpolation for the lower-bound formulation described below.

3.1. Linear constraints

3.1.1. Equilibrium inside each element

We first consider the continuum stress equilibrium equations:

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \tag{19}$$

where $\mathbf{b}$ denotes the body force vector. For the linear triangular element adopted in this work, the stress field is interpolated using the standard linear shape functions defined in terms of area coordinates a_1, a_2, a_3 or volume coordinates v_1, v_2, v_3, v_4 :

$$\mathbf{h}^{(a,1)} = \{a_1, a_2, a_3\} \quad \text{or} \quad \mathbf{h}^{(v,1)} = \{v_1, v_2, v_3, v_4\}. \quad (20)$$

If the stress values at all nodes (stress points) are given, the stress value at a point with area coordinates $\mathbf{a}$ in an element can be interpolated using:

$$\boldsymbol{\sigma} = \sum_{i=1}^3 \mathbf{h}_i^{(a,1)} \boldsymbol{\sigma}_i^* \quad (21)$$

in 2D, and:

$$\boldsymbol{\sigma} = \sum_{i=1}^4 \mathbf{h}_i^{(v,1)} \boldsymbol{\sigma}_i^* \quad (22)$$

in 3D.

For the linear triangular element used here, $N_{\text{sp,e}} = 3$ in 2D, corresponding to the three vertices of each element. The formulation can be extended to 3D tetrahedral linear elements in an analogous manner. The vector $\boldsymbol{\sigma}_i^*$ contains the Cartesian components of the stress tensor of the i -th stress point as:

$$\boldsymbol{\sigma}_i^* = \begin{cases} [\sigma_{xx,i}^*, \sigma_{yy,i}^*, \sigma_{xy,i}^*]^\top, & \text{in 2D,} \\ [\sigma_{xx,i}^*, \sigma_{yy,i}^*, \sigma_{zz,i}^*, \sigma_{xy,i}^*, \sigma_{xz,i}^*, \sigma_{yz,i}^*]^\top, & \text{in 3D,} \end{cases} \quad i \leq N_{\text{sp}}. \quad (23)$$

Here N_{sp} is the number of total stress points in the problem.

For strict intra-element equilibrium, the stress field within each linear element must satisfy:

$$\mathbf{E} \tilde{\boldsymbol{\sigma}}^* + \mathbf{b} = \mathbf{0} \quad (24)$$

throughout the element, where $\tilde{\boldsymbol{\sigma}}^*$ contains the Cartesian components of the stress tensor associated with the stress points of the element, $\mathbf{b} = [b_x, b_y]^\top$ denotes the body force vector, and $\mathbf{E}$ is the element equilibrium matrix. For the linear triangular element adopted here, $\mathbf{E}$ is constructed by arranging the spatial derivatives of the linear basis functions $\mathbf{h}_i^{(a,1)}$:

$$\mathbf{E} = \begin{bmatrix} h_{1,x}^{(a,1)} & 0 & h_{1,y}^{(a,1)} & \dots & h_{3,x}^{(a,1)} & 0 & h_{3,y}^{(a,1)} \\ 0 & h_{1,y}^{(a,1)} & h_{1,x}^{(a,1)} & \dots & 0 & h_{3,y}^{(a,1)} & h_{3,x}^{(a,1)} \end{bmatrix} \quad (25)$$

where the subscripts x and y denote derivatives with respect to the spatial coordinates. Since the first-order basis functions $h_i^{(a,1)}$ are linear in the area coordinates, their spatial derivatives are constant within each element. Therefore, the equilibrium matrix $\mathbf{E}$ remains constant over the element.

3.1.2. Equilibrium between adjacent elements

In LB FELA, statically admissible discontinuities between adjacent elements are often introduced to improve the solution quality. The traction vector at the i th stress point on the interface of element A can be obtained by:

$$\mathbf{t}_{Ai}^* = \mathbf{E}_A \boldsymbol{\sigma}_{Ai}^* \quad (26)$$

where $\boldsymbol{\sigma}_{Ai}^*$ is the stress tensor of the i th stress point of element A on the interface, as described in Equation (23). The matrix $\mathbf{E}_A$ projects the stress tensor in Cartesian coordinates onto the local interface surface. In 2D, this is specified by:

$$\mathbf{E}_A = \begin{bmatrix} n_x & 0 & n_y \\ 0 & n_y & n_x \end{bmatrix} \quad (27)$$

In 3D, this matrix is further specified by:

$$\mathbf{E}_A = \begin{bmatrix} n_x & 0 & 0 & n_y & n_z & 0 \\ 0 & n_y & 0 & n_x & 0 & n_z \\ 0 & 0 & n_z & 0 & n_x & n_y \end{bmatrix}. \quad (28)$$

where n_x , n_y and n_z are the components of the unit normal to the interface. For strict equilibrium across the interface, the traction vectors of element A and element B , namely, $\mathbf{t}_A$ and $\mathbf{t}_B$, must be equal and opposite at every stress point on the interface, and therefore it is sufficient to enforce:

$$\mathbf{E}_A \boldsymbol{\sigma}_{Ai}^* + \mathbf{E}_B \boldsymbol{\sigma}_{Bi}^* = \mathbf{0} \quad \forall i \in \{1, \dots, p+1\} \quad (29)$$

3.1.3. Boundary conditions

For a given boundary edge Γ_t , the prescribed traction is described as:

$$\boldsymbol{\sigma} \mathbf{n} = \bar{\mathbf{t}} \quad \text{on } \Gamma_t. \quad (30)$$

3.2. Yield criterion

3.2.1. Mohr-Coulomb failure criterion in 2D

The use of Bernstein elements, as described in the previous section, guarantees a strict lower bound as long as the failure criterion is satisfied at all stress points within the domain.

In plane strain problems, the Mohr-Coulomb failure criterion for a stress point can be expressed as:

$$\sqrt{\left(\frac{\sigma_{xx,i} - \sigma_{yy,i}}{2}\right)^2 + \sigma_{xy,i}^2} + \frac{(\sigma_{xx,i} + \sigma_{yy,i})}{2} \sin \phi \leq c \cos \phi \quad (31)$$

Specifically, when $\phi = 0$, this reduces to the Tresca failure criterion:

$$\sqrt{\left(\frac{\sigma_{xx,i} - \sigma_{yy,i}}{2}\right)^2 + \sigma_{xy,i}^2} \leq c \quad (32)$$

In implementing the Mohr-Coulomb failure criterion described in (31), we introduce an affine transformation matrix as described in [23], that allows the criterion to be expressed at stress point i in the uniform $Ax + s = b$ manner as follows:

$$\begin{bmatrix} \frac{\sin \phi}{2} & \frac{\sin \phi}{2} & 0 \\ \frac{1}{2} & -\frac{1}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \sigma_{xx,i} \\ \sigma_{yy,i} \\ \sigma_{xy,i} \end{bmatrix} + \begin{bmatrix} s_{1,i} \\ s_{2,i} \\ s_{3,i} \end{bmatrix} = \begin{bmatrix} c \cos \phi \\ 0 \\ 0 \end{bmatrix} \quad (33)$$

Then the Mohr-Coulomb failure criterion on stress point i can be satisfied by requiring the second-order cone (SOC) constraint:

$$\|(s_{2,i}, s_{3,i})\|_2 \leq s_{1,i} \quad (34)$$

3.2.2. Drucker-Prager criterion in 3D

In 3D problems, our current implementation is now able to handle the Drucker-Prager criterion in SOCP form, as described in [23]. In Cartesian coordinates, this criterion for a stress point may be expressed as:

$$\sqrt{J_2} + \alpha \sigma_m - k \leq 0 \quad (35)$$

here, J_2 is the second invariant of the deviatoric stress tensor, σ_m is the mean stress, and α and k are the material parameters.

This can be converted into a SOCP formulation by introducing the affine transformation matrix:

$$\begin{bmatrix} \alpha & \alpha & \alpha & 0 & 0 & 0 \\ \frac{1}{2} & 0 & -\frac{1}{2} & 0 & 0 & 0 \\ -\frac{\sqrt{3}}{6} & \frac{\sqrt{3}}{3} & -\frac{\sqrt{3}}{6} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \sigma_{xx,i} \\ \sigma_{yy,i} \\ \sigma_{zz,i} \\ \tau_{xy,i} \\ \tau_{xz,i} \\ \tau_{yz,i} \end{bmatrix} + \begin{bmatrix} s_{1,i} \\ s_{2,i} \\ s_{3,i} \\ s_{4,i} \\ s_{5,i} \\ s_{6,i} \end{bmatrix} = \begin{bmatrix} k \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (36)$$

Then the Drucker-Prager failure criterion on stress point i can be satisfied by requiring the SOC constraint:

$$\|(s_{2,i}, s_{3,i}, s_{4,i}, s_{5,i}, s_{6,i})\|_2 \leq s_{1,i} \quad (37)$$

3.3. FELA-specific SOCP structure

With knowledge of the aforementioned constraints, we may implement the intra/inter-element equilibrium, and traction boundary conditions as linear equilibrium constraints, and cast the yield criteria both in 2D and 3D for each stress point as SOC constraints. Then the LB-FELA discretization can be readily cast into large-scale SOCPs using a standard conic form as stated in 1.

For both the 2D Mohr–Coulomb and 3D Drucker–Prager formulations, the slack vector associated with stress point i can be written in the unified form

$$\mathbf{s}_i = \begin{bmatrix} t_i \\ \mathbf{r}_i \end{bmatrix}, \quad t_i = s_{1,i}, \quad (38)$$

where

$$\mathbf{r}_i = \begin{cases} [s_{2,i}, s_{3,i}]^\top, & \text{in 2D,} \\ [s_{2,i}, s_{3,i}, s_{4,i}, s_{5,i}, s_{6,i}]^\top, & \text{in 3D.} \end{cases} \quad (39)$$

Accordingly, the yield constraint at each stress point takes the standard SOC form

$$\|\mathbf{r}_i\|_2 \leq t_i. \quad (40)$$

The cone associated with the complete LB-FELA problem is therefore the Cartesian product of a zero cone corresponding to the linear equality constraints and a collection of local SOC blocks, which can be readily mapped to the feasible region defined in Equation (2). In the present formulation, the linear equilibrium and boundary conditions are enforced by fixing the corresponding slack components to zero, and the dimension for the 2D Mohr-Coulomb criterion $q_i = 3$ and $q_i = 6$ for the 3D Drucker-Prager criterion, while each SOC block $\mathcal{Q}^3$ represents a yield constraint of the form $\|(s_{2,i}, s_{3,i})\|_2 \leq s_{1,i}$ in 2D FELA. In 3D, each SOC block $\mathcal{Q}^6$ represents a yield constraint of the form $\|(s_{2,i}, s_{3,i}, s_{4,i}, s_{5,i}, s_{6,i})\|_2 \leq s_{1,i}$.

The zero-cone component enforces the equilibrium and traction boundary conditions by fixing the corresponding slack variables to zero, whereas the SOC blocks enforce the local yield admissibility conditions. This block structure allows the conic projection required by the ADMM scheme to be performed independently for each stress point.

3.4. Adaptive mesh refinement scheme

Adaptive mesh refinement (AMR) has been widely adopted in FELA due to its ability to produce tighter lower and upper bounds while alleviating computational demands [29, 30, 31, 32, 33, 15].

Specifically, Ciria [31] also incorporated simple heuristics such as regular refinement and longest-edge bisection, refining the elements that contributed most to the error bound gap. Lyamin [30] proposed an adaptive remeshing strategy for lower bound analysis that uses an advancing-front mesh generator, which is able to capture localized plastic zones near features like footing edges. Lagrange multipliers were chosen as the control variable for error estimation, and various adaptive strategies were compared against each other on numerical examples such as the stability of a vertical cut and the bearing capacity of a strip footing. Martin demonstrated how the robust general-purpose 2D Delaunay triangular mesh generator Triangle [34] could be used together with the author’s in-house FELA code OxLim to resolve slip-line fields [33]. The adaptively refined mesh and the slip line solution show good agreement. Sun [32] adopted Tetgen [35] for adaptive meshing in 3D, and came up with a methodology of using only the LB solution.

Among these approaches, Lyamin [30] and Sun [32] identify elements for refinement using indicators derived solely from a lower-bound (LB) solution. In contrast, Martin [33] and da Silva [15] rely on the strain-rate field from

an upper-bound (UB) solution, marking elements so as to approximately equalize the quantity $\int \gamma_{\max} dA$ across the mesh.

Apart from that, the warm-starting capability of ADMM-based solvers was exploited more thoroughly, since the novel mesh is largely based on the previous mesh and the variables may be copied directly to initialize the solver. This, in practice, resulted in a significant reduction in the number of iterations needed in each computation.

We used Triangle [34] to adaptively refine the unstructured mesh in 2D, and used Tetgen [35] in 3D. The refinement indicator adopted here follows Sun [32], enabling mesh adaptivity using only the LB solution from the previous iteration. We employ a stress-indicator-based AMR strategy for both the Tresca and Mohr–Coulomb criteria. The procedure is summarized as follows:

1. Solve the coarse-mesh LB-FELA problem to obtain a converged ADMM iterate (x, s, y) .
2. With the SOC slack variable s retained at each stress point, we define a scalar stress indicator that measures proximity to yield. Recall that the yield constraint is written as $\|\mathbf{r}_i\|_2 \leq t_i$. We define the stress indicator:

$$\eta_i := \frac{\|\mathbf{r}_i\|_2}{t_i}. \quad (41)$$

In our setting with $c > 0$ and $\phi < 90^\circ$, the SOC formulation yields $s_{1,i} > 0$ in practice; hence (41) is well-defined. Larger η_i indicates a stress state closer to the yield surface and is therefore prioritized for refinement.

3. For each element e , compute an element-wise indicator by taking the maximum over its stress points,

$$\eta_e := \max_{i \in \mathcal{S}(e)} \eta_i, \quad (42)$$

where $\mathcal{S}(e)$ denotes the set of stress points associated with element e .

4. Partially sort $\{\eta_e\}$ and mark the top $n = \mu N_{\text{elem}}$ elements for refinement, where N_{elem} is the total number of elements and $\mu \in (0, 1)$ controls the refinement fraction. We use $\mu = 0.2$ by default.
5. Generate a `.area` file for the mesh generator Triangle [34]. Elements that are not refined are assigned -1. For each marked element e , we

assign a target maximum area

$$A_{\max,e}^{\text{new}} = \frac{A_e}{r}, \quad (43)$$

where A_e is the current area of element e and $r > 1$ is the refinement ratio. We use $r = 4$ by default.

4. Acceleration techniques

4.1. Coarse-to-fine continuation via warm starts

To reduce the time-to-solution on fine discretizations, we adopt a coarse-to-fine continuation strategy that exploits the warm-start capability of ADMM. After solving the SOCP on a coarse mesh, we store (i) the coarse mesh geometry and connectivity, and (ii) the converged ADMM variables from the previous iteration (x^k, s^k, y^k) . When a refined mesh is generated, these quantities are transferred to initialize the solver on the refined discretization. The same strategy applies to both two- and three-dimensional meshes; the only difference is that the simplex element is a triangle in 2D and a tetrahedron in 3D.

Containing element identification with an AABB tree. A key step in the transfer is to determine, for each point on the refined mesh, the containing element in the coarse mesh. We accelerate this search using an axis-aligned bounding box (AABB) tree built over the coarse elements. Given a query point's coordinates as input, the AABB tree efficiently identifies candidate coarse elements, after which a point-in-element test is used to determine the containing coarse element. This procedure is illustrated in 2D in Figure 1; the same approach can be extended straightforwardly to tetrahedral elements in 3D.

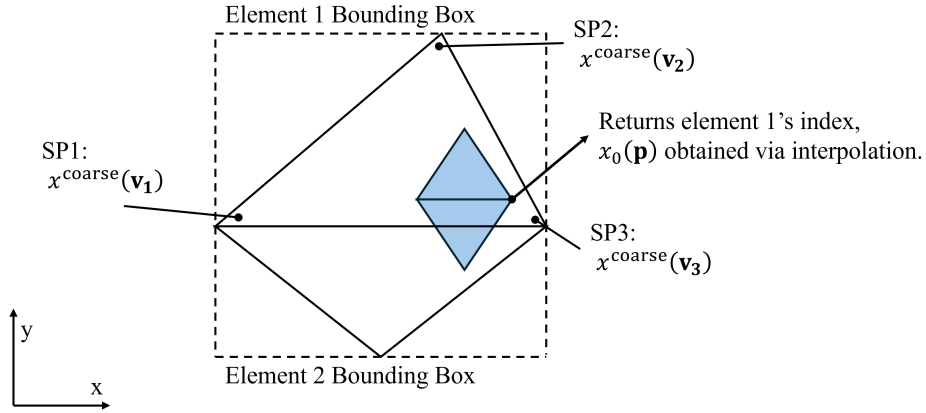


Figure 1: How AABB locates the containing element of a given query point

Element-level initialization. For each refined element, we first compute its centroid and identify the corresponding parent element in the coarse mesh using the AABB tree. Element-associated quantities, such as the dual variables associated with intra-element equilibrium constraints, are then mapped from the parent coarse element to the refined element.

Stress-point interpolation via barycentric coordinates. We then initialize the variables at the stress points of each refined element. For a given stress point $\mathbf{p}$ located inside a coarse simplex element with vertices $\{\mathbf{v}_j\}_{j=1}^{n_v}$, we compute its barycentric coordinates $\{\lambda_j\}_{j=1}^{n_v}$, where n_v , the number of vertices for an element, $n_v = 3$ for a triangle in 2D and $n_v = 4$ for a tetrahedron in 3D. The warm-start value is then obtained by interpolation:

$$x_0(\mathbf{p}) = \sum_{j=1}^{n_v} \lambda_j x^{\text{coarse}}(\mathbf{v}_j), \quad (44)$$

with $s_0(\mathbf{p})$ and $y_0(\mathbf{p})$ initialized analogously. This formulation provides a dimension-independent transfer rule and allows higher-order discretizations to be warm-started without requiring a prior higher-order solution; in practice, a converged linear-element solution is sufficient.

4.2. Active cone identification

ADMM naturally suffers from its linear convergence, and its performance largely depends on the choice of the penalty parameter in the augmented Lagrangian [18]. Following the adaptive penalty updating strategy in [18], we extend it to SOC constraints by exploiting information from the projection step.

For each SOC block, we consider the slack vector $\mathbf{s}_i$. During projection, the block is mapped either onto the cone apex or onto the boundary, that is, when $\|\mathbf{r}_i\| \leq -t_i$ or $\|\mathbf{r}_i\| > t_i$. We consider the i th cone as active whenever either of these conditions holds. After identifying the active cones, we choose to update the diagonal penalty matrix $D(\rho)$ every 20 iterations. This procedure is further illustrated in Figure 2

A fixed bound B^k is imposed to limit the value of the entries of $D(\rho)$ to prevent the system from becoming ill-conditioned. We denote the j -th entry of the diagonal matrix $D(\rho)$ as $D(\rho)_{j,j}^k$, we clamp all entries associated with the SOC blocks to the interval:

$$1/B^k \leq D(\rho)_{j,j}^k \leq B^k \quad (45)$$

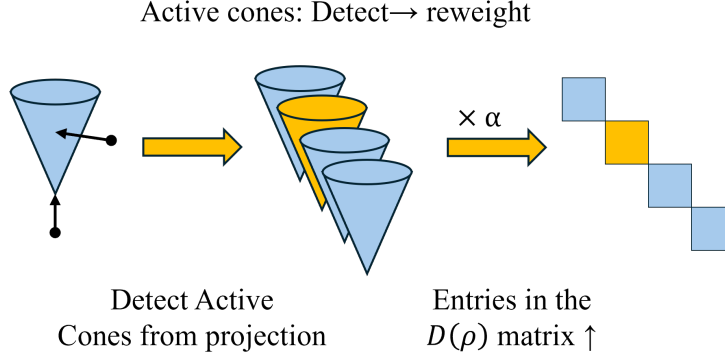


Figure 2: Active cone identification scheme

Since the penalty is associated with each SOC block, any update to the $D(\rho)$ matrix, must scale all diagonal entries corresponding to the same cone to maintain consistent enforcement. If the cone is active, we multiply all the associated entries by α , otherwise do nothing.

$$D(\rho)_{j,j}^{k+1} = \begin{cases} \alpha D(\rho)_{j,j}^k, & \text{if } \|\mathbf{r}_i\| \leq -t_i \text{ or } \|\mathbf{r}_i\| > t_i \\ D(\rho)_{j,j}^{(k)}, & \text{otherwise,} \end{cases} \quad (46)$$

Also, at the first 200 iterations, we enforce $\alpha = 2$ when the active set is uncertain at the first 200 iterations. After the stabilization phase, we set $\alpha = 10$ for a more aggressive update. Also, we implement a simple yet effective adaptive adjustment mechanism of the value α near convergence, the heuristic is to reduce α smoothly via a quadratic rule as the iterates approach convergence, avoiding abrupt changes that could destabilize the system or perturb the final solution.

The above updates only happen if the system satisfies the following criteria:

$$r_{\text{dual}} \leq 0.1 \varepsilon_{\text{dual}} \quad (47)$$

i.e., the dual feasibility condition is already well satisfied, whereas the primal residual remains far from convergence. In practice, with a reasonable initial choice of ρ , the dual residual typically decreases to its tolerance earlier than the primal residual.

These simple heuristics prevent the $D(\rho)$ matrix from changing drastically when approaching convergence, while preserving a robust way of updating

the variables in a way that the primal and dual residuals are balanced. The idea of balancing these two residuals is inspired by the OSQP solver [17], which also adopted an adaptive ρ measure, however the parameter ρ updated uniformly across all constraints, whereas here we adaptively update the penalty only on the active conic blocks.

Notice that the linear constraints are unchanged during the process. Updates to the diagonal weighting matrix $D(\rho)$ modify the system matrix $A^\top D(\rho)A$. Importantly, since $D(\rho)$ only rescales constraint rows, these updates do not change the sparsity pattern of $A^\top D(\rho)A$. We therefore reuse the symbolic factorization and perform only a numerical refactorization of $A^\top D(\rho)A$ whenever $D(\rho)$ is updated, reducing the overhead of refactorization.

4.3. Augmented ADMM formulation

In the GPU-accelerated 3D FELA implementation, we found that even medium-sized problems can exceed the capabilities of a relatively modest GPU (in our case, an NVIDIA RTX A1000). In practice, we observed that in 3D FELA, the condensed matrix becomes increasingly dense as the problem size grows, while the ratio of $\frac{A^\top D(\rho)A.\text{nnz}()}{A.\text{nnz}()}$ remained close to constant in 2D. This motivated us to solve the augmented linear system rather than the condensed system, with the expectation that the augmented form would lead to fewer non-zero fill-ins during factorization. In practice, we always solve the augmented linear system in 3D.

To improve the conditioning of the linear system, a regularization parameter σ is introduced. Although the augmented system is no longer positive definite, it remains quasi-definite and therefore admits a well-defined $LDL^\top$ factorization [19]:

$$\begin{bmatrix} \sigma I & A^\top \\ A & -D(\rho)^{-1} \end{bmatrix} \begin{bmatrix} x^{k+1} \\ v \end{bmatrix} = \begin{bmatrix} -c + \sigma x^k \\ b - s - D(\rho)^{-1}y \end{bmatrix} \quad (48)$$

4.4. Alternative ellipsoidal projection scheme

Solver performance may also be improved by adopting an alternative projection strategy. In MOSEK and in our ADMM implementation, the projection step is introduced by adding an additional set of variables s and enforcing $Ax + s = b$, where, for each conic constraint, A is the affine transformation matrix defined in (33). When $\alpha = 0$, the Drucker–Prager criterion

reduces to the von Mises criterion, and the corresponding SOC projection becomes equivalent to a projection onto an ellipsoid.

This ellipsoidal projection scheme was implemented only in the 3D programme. After splitting and enforcing $x = s$, the projection step at each stress point i amounts to projecting the associated slack vector

$$s_i^{\text{trial}} = (s_{1,i}, s_{2,i}, \dots, s_{6,i})^\top$$

onto the ellipsoid

$$\mathcal{E} = \{ s \in \mathbb{R}^6 \mid s^\top P s \leq 1 \}.$$

The projection problem can therefore be written as

$$\begin{aligned} & \textbf{minimize} && \frac{1}{2} \|s - s_i^{\text{trial}}\|^2 \\ & \textbf{subject to} && s^\top P s \leq 1, \end{aligned} \tag{49}$$

where s_i^{trial} denotes the current slack vector before projection.

For stress points lying outside the ellipsoid, the inequality constraint is active at the solution, and the KKT conditions are

$$\begin{aligned} 1) \quad & s - s_i^{\text{trial}} + 2\lambda P s = 0, \\ 2) \quad & s^\top P s = 1, \\ 3) \quad & \lambda \geq 0. \end{aligned} \tag{50}$$

Let

$$\mu = 2\lambda \geq 0.$$

Then we obtain

$$(I + \mu P)s = s_i^{\text{trial}}, \tag{51}$$

and hence

$$s(\mu) = (I + \mu P)^{-1} s_i^{\text{trial}}. \tag{52}$$

We now give the formulation of the P matrix as:

$$P = \frac{1}{k^2} \begin{bmatrix} \frac{1}{3} & -\frac{1}{6} & -\frac{1}{6} & 0 & 0 & 0 \\ -\frac{1}{6} & \frac{1}{3} & -\frac{1}{6} & 0 & 0 & 0 \\ -\frac{1}{6} & -\frac{1}{6} & \frac{1}{3} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \tag{53}$$

Substituting $s(\mu)$ into the active constraint yields the scalar root-finding problem

$$\phi(\mu) = s(\mu)^\top P s(\mu) - 1 = 0, \quad (54)$$

which can be solved efficiently by Newton’s method:

$$\mu_{j+1} = \mu_j - \frac{\phi(\mu_j)}{\phi'(\mu_j)}. \quad (55)$$

In practice, although a large number of stress points must be projected, the high-level parallelisation provided by a customised CUDA kernel reduces the cost of this step to only about 3% of the total solution time.

4.5. Implementation of the solver on CUDA

Profiling of the code on CPU showed that apart from the obvious bottleneck of the solve operation that took place in every iteration, the various sparse matrix manipulations, if not handled efficiently, may also significantly degrade overall solver performance. To address this, the implementation makes intensive use of GPU acceleration, and the following improvements were made:

cuDSS for solving the linear system. In the condensed formulation, the KKT system is symmetric positive-definite. As a result, cuDSS internally uses a Cholesky factorization path, which is faster and more memory-efficient than the general-purpose $LDL^\top$ decomposition. This choice leads to significant runtime improvements in the solve phase of each ADMM iteration in 2D FELA, however, in 3D, when memory usage of a condensed formulation exceeds the capacity of the GPU, the augmented formulation, even though it used a sparse $LDL^\top$ decomposition, was faster in time per-iteration.

cuBLAS and cuSPARSE for efficient construction of vectors. cuBLAS is a CUDA implementation of BLAS, which enables convenient access to GPU-accelerated BLAS operations. cuSPARSE is a CUDA library that contains many commonly used matrix manipulation subroutines.

CUDA kernel functions for customized variable updates. CUDA kernel functions are special C functions executed on the GPU in a highly parallelised fashion [36]. Each thread runs the same instructions on different data, making them inherently suitable for SIMD-style computations. While many standard linear algebra operations are efficiently handled by highly optimized

libraries such as cuBLAS and cuSPARSE, several complex or non-standard operations required in our solver are not directly supported.

For example, the non-uniform scaling of the variables and matrices, and simultaneous projection onto N_{SOC} second-order cones, and parallel updates of intermediate vectors fall outside of the scope of existing CUDA libraries.

All operations completed on GPU. Though the construction of the KKT matrix and warm-starting is completed outside of the GPU, all the subsequent ADMM iterations are completed on the GPU, only passing the primal and dual residual and the current objective back to the CPU. This design significantly eliminated the overhead introduced by the frequent GPU-CPU data transfers.

5. Numerical Experiments

In this section, we present 3 sets of classical FELA benchmark problems to assess the performance of the proposed ADMM solver in comparison with two state-of-the-art commercial interior-point solvers, MOSEK and Gurobi.

In each of the following examples, we first describe the problem setup and then present numerical results to assess the accuracy, robustness, and scalability of the proposed ADMM solver in comparison with IPM codes. For both the 2D plane-strain and 3D cases, three benchmark problems are considered: the rigid platen, strip footing, and vertical cut problems. For each benchmark, the computational performance of the proposed ADMM solver is compared with that of commercial interior-point solvers in terms of solution time and accuracy. Adaptive mesh refinement results are also presented to demonstrate the FELA formulation’s ability to capture localized failure mechanisms efficiently.

5.1. Experimental setup

Linear stress elements were used in the comparison of accuracy and speed of the in-house solver and IPM solvers in all three geotechnical problems.

All CPU code was run on an Intel Core i7-12700H, and all GPU code were run on a NVIDIA RTX A1000. We assume Tresca soil with $c = 1$ on 2D problems, and von Mises soil with $k = 1$ on 3D problems. The GPU implementation was developed using NVIDIA CUDA 12.8 and cuDSS 0.5.0. Its performance was compared with MOSEK 11.0 and Gurobi 12.0.2, which served as the commercial interior-point method baselines. In all experiments, MOSEK and Gurobi were run using their default termination criteria on 14 CPU cores.

5.2. 2D Plane Strain Benchmark Problems

5.2.1. Rigid Platen Problem

Problem setup. In this problem, a block of Tresca soil of size $L \times 0.5L$ is subjected to compression between two rigid platens. The interfaces between the platens and the soil are assumed to be fully rough, such that no constraints need to be placed on the interface stresses. Symmetric boundary conditions are applied to the left and bottom boundaries, while the right boundary is left free. An illustrative representation of the physical setup is provided in Figure 3.

The exact solution of the load multiplier for the rigid platen problem considered here, $\beta = 2.42768$, was first given by Salençon [12, 37].

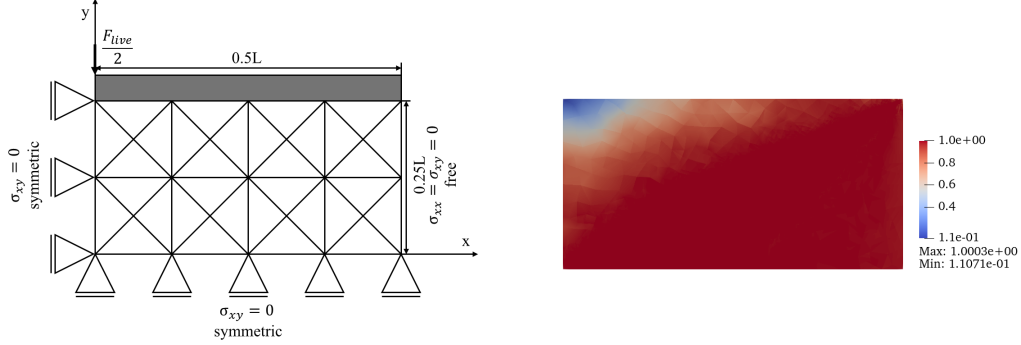


Figure 3: Rigid platen problem. **Left:** physical configuration; **Right:** obtained yield function field using AMR with linear stress elements

Benchmark comparison. The proposed ADMM solver is benchmarked against the commercial interior-point solvers MOSEK and Gurobi using structured meshes of linear stress elements with progressively refined discretizations.

Table 1 shows that the proposed solver delivers superior computational performance, achieving a speedup of $4\times$ to $10\times$ over the state-of-the-art IPM solvers, while maintaining high solution accuracy in all cases. For each mesh discretization, all three solvers converge to closely comparable collapse multipliers.

Figure ?? shows how our solver compared with MOSEK on 2D problems, the problem sizes are reported in Table 1, ranging from 0.18 million variables to 1.84 million variables.

The computed collapse multipliers remain below the analytical solution reported by Salençon [12, 37]. This gap is primarily due to the use of a structured mesh. Using higher-order stress elements may mitigate this effect and yield a tighter lower bound. We nonetheless adopt structured meshes in this example to ensure controlled and consistent problem generation, which is desirable for a fair performance comparison. Overall, the results reported here demonstrated that the proposed ADMM methodology works well on these types of PDE-constrained optimization problem.

Adaptive mesh refinement solution. Using AMR, we obtain a tighter lower bound solution of $\beta = 2.426787$ using linear stress elements, together with AMR, the resulting yield function is shown in Figure 3, with the final adaptively refined mesh after 10 refinements shown in Figure 4. This gives a relative error of only 0.036%, the refined mesh concentrated around the yielded

Table 1: Solver performance comparison on the 2D rigid platen benchmark for different mesh discretizations

Problem size		Solver	Solve time (s)	Collapse multiplier	Iterations	Time-per-iter (s)
$N_{\text{elem}}(\times 10^3)N_{\text{var}}(\times 10^6)$						
20.0	0.18	MOSEK	8.80	2.42465	32	0.37
		Gurobi	6.96	2.42465	26	0.28
		ADMM	2.66	2.42467	340	0.0078
28.8	0.26	MOSEK	31.75	2.42514	35	0.91
		Gurobi	13.98	2.42514	30	0.68
		ADMM	3.62	2.42516	260	0.014
51.2	0.46	MOSEK	60.03	2.42576	46	1.11
		Gurobi	27.60	2.42577	34	1.31
		ADMM	5.74	2.42577	300	0.019
80.0	0.72	MOSEK	43.14	2.42614	38	1.14
		Gurobi	50.81	2.42614	27	2.48
		ADMM	9.82	2.42611	340	0.029
204.8	1.84	MOSEK	95.24	2.42669	39	3.024
		Gurobi	342.17	2.42671	32	12.01
		ADMM	20.36	2.42669	280	0.073

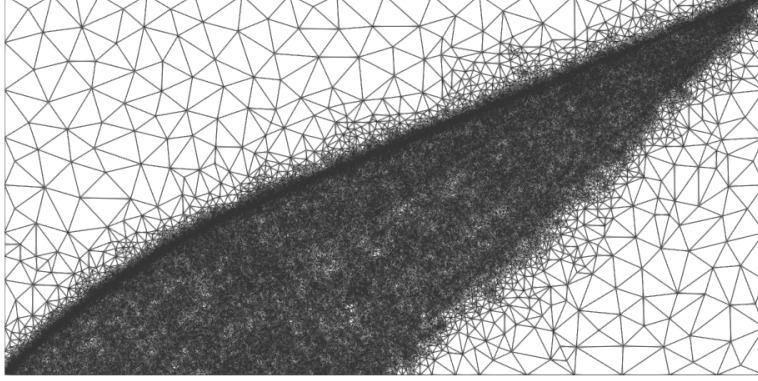


Figure 4: AMR result of the rigid platen problem

area. Moreover, the zone that had been refined the most agrees well with the solutions of da Silva [12] and with the classical slip line solutions [38].

5.2.2. Strip Footing Problem

Problem setup. An illustrative representation of the problem is shown in Figure 5. Here, a rigid body is placed at the upper-left corner of the domain to represent a strip footing. A fixed boundary condition is applied on the right and bottom edges, and a symmetric boundary condition on the left edge. The fixed boundaries are sufficiently far from the footing to have a minimal effect on the result.

The ultimate bearing capacity of a strip footing on weightless cohesive soil is a foundational result in geotechnical engineering. The exact solution of the load multiplier to this problem is $\beta = 2 + \pi$, a result first derived by Prandtl [39, 23, 40]. Martin developed software ABC for calculating the ultimate bearing capacity of this footing using the method of characteristics [41]. A FELA-based solution to this problem with adaptive mesh refinement was first given by Lyamin [30], demonstrating that the approach can deliver accurate collapse loads while systematically assessing a range of adaptive remeshing strategies.

Benchmark comparison. We first demonstrate the performance of our 2D condensed-form ADMM solver on different mesh discretizations, all using linear stress elements.

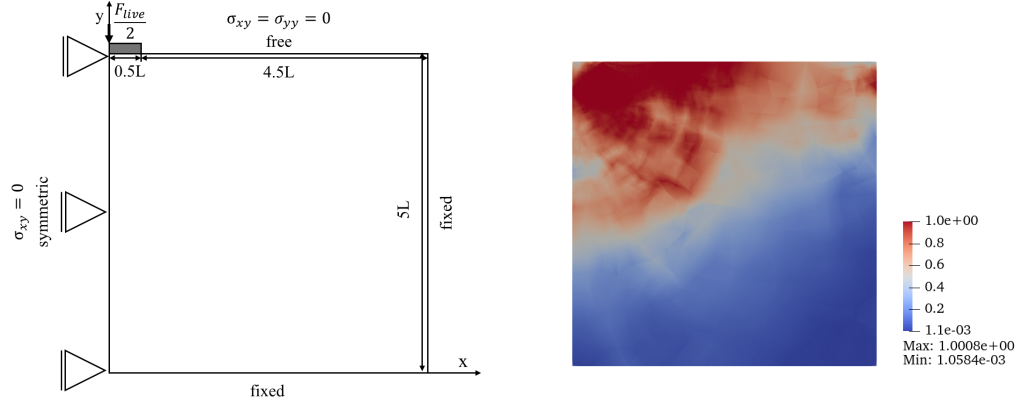


Figure 5: Strip footing problem. **Left:** physical configuration; **Right:** solution of the yield function using AMR and linear stress elements

Table 2: Solver performance comparison on the 2D strip footing benchmark for different mesh discretization

Problem size		Solver	Solve time (s)	Collapse multiplier	Iterations	Time-per-iter (s)
$N_{\text{elem}} (\times 10^3) N_{\text{var}} (\times 10^6)$						
40.0	0.36	MOSEK	27.37	5.09752	32	0.86
		Gurobi	24.73	5.09753	231	1.09
		ADMM	6.08	5.09768	440	0.014
57.6	0.52	MOSEK	48.59	5.10470	38	1.28
		Gurobi	26.78	5.10470	27	1.26
		ADMM	8.47	5.10482	440	0.019
102.4	0.92	MOSEK	204.05	5.11383	50	5.57
		Gurobi	173.61	5.11383	35	4.08
		ADMM	13.77	5.11392	400	0.034
160.0	1.44	MOSEK	126.85	5.11932	33	3.84
		Gurobi	270.60	5.11932	35	8.59
		ADMM	33.21	5.11945	620	0.054

Table 2 shows that the proposed GPU-based ADMM solver remains faster than the CPU-parallelized IPM solver MOSEK and Gurobi, while producing almost identical collapse multipliers. The observed speedup ranges from $3.12\times$ to $5.7\times$ for MOSEK, and $6.04\times$ to $17.28\times$ for Gurobi. Although this improvement is less pronounced than that obtained for the 2D rigid platen problem, it still demonstrates the efficiency of the proposed solver on classical FELA problems.

The final load multiplier remains below Prandtl’s analytical solution of $2 + \pi$, with an error of only 0.27%. This discrepancy is mainly due to the use of linear stress elements on structured meshes for benchmarking. A tighter LB solution is obtained below using AMR and linear stress elements:

Adaptive mesh refinement solution. Here we demonstrate an ADMM–FELA framework enhanced by AMR using linear stress elements. By mapping the solution from the previous mesh onto the refined mesh as the initial iterate, we achieve a substantial reduction in iterations while maintaining accuracy. The final yield function field, after 9 refinements, is given in Figure 5. Solving this problem yields a load multiplier of $\beta = 5.13648$, while only using 58.5 seconds in its final iteration. The obtained load multiplier agrees well with the analytical solution by Prandtl [40, 23].

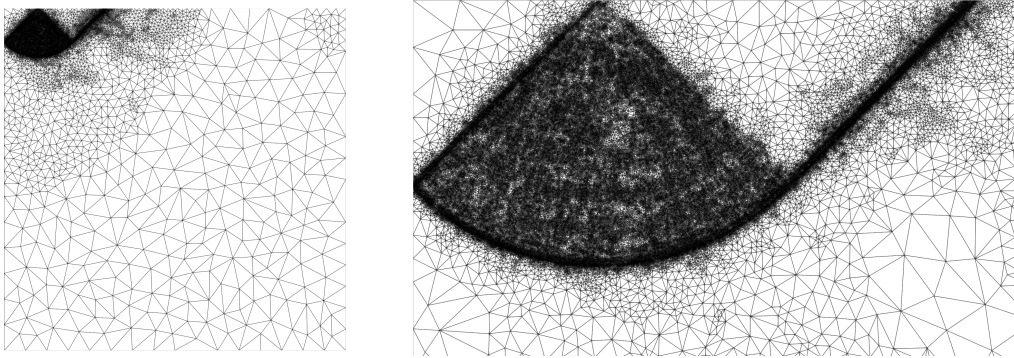


Figure 6: AMR mesh of the strip footing problem in plane strain. **Left:** overall obtained mesh; **Right:** zoomed view highlighting the refined area

The resulting AMR mesh shown in Figure 6 is consistent with the sequential limit-analysis meshes reported by Kong [42] and Martin [41, 33], in that the refined region correctly matches the slip-line zone.

This example also highlights the overall efficiency of the proposed solver, which attains rapid convergence on large-scale problems. Our solver con-

verges easier due to exploiting the problem structure by warm-starting each AMR solve with the solution from the previous mesh.

5.2.3. Vertical Cut Problem

Problem setup. We consider a square soil block with a vertical excavation on its left side, subjected to a uniform body force representing a live load. The top surface and the excavated left boundary are assumed to be free, while the right and bottom boundaries are taken as rough and fixed. An illustrative representation of the problem and a numerical solution given by using AMR is shown in Figure 7. This benchmark is of particular interest because the uniform body force introduces a dense column into the matrix A , which can pose a challenge for solver performance.

Although, to the best of the authors' knowledge, no analytical solution has been reported in the literature, Martin [33] provided a tight bound on the ultimate bearing capacity, $3.7764904 < N < 3.7764911$, using finite element limit analysis with adaptive mesh refinement. The corresponding slip-line field was also obtained using this approach.

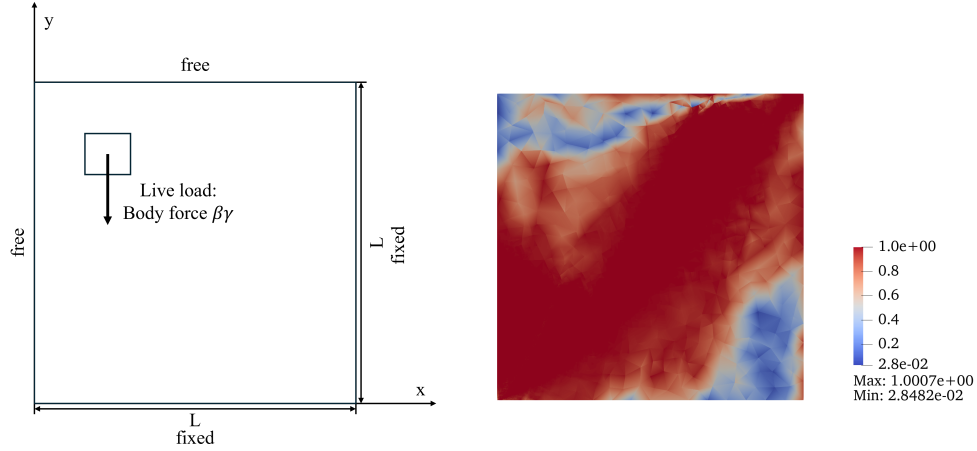


Figure 7: Vertical cut problem. **Left:** physical configuration; **Right:** obtained yield function field

Benchmark Comparison. We first assess the performance of the proposed ADMM solver using a sequence of increasingly refined meshes using linear stress elements.

As shown in Table ??, the proposed ADMM solver constantly outperforms the IPM solvers in problems with dimension ranging from 0.36 to 1.44 million

variables. ADMM attained a speedup from $6.6\times$ to $17.6\times$ compared with MOSEK, and from $6.0\times$ to $17.3\times$ with Gurobi. Even with the improvements made to speed up convergence, the problems still takes around 300 to 500 iterations, but each iteration is considerably cheaper than the IPM iterations. Also, the ADMM solver attained the speedup while preserving accuracy, obtaining a nearly identical load multiplier with the accurate IPM solvers.

Table 3: Solver performance comparison on the 2D vertical cut benchmark for different mesh discretization

Problem size		Solver	Solve time (s)	Collapse multiplier	Iterations	Time-per-iter (s)
$N_{\text{elem}}(\times 10^3)N_{\text{var}}(\times 10^6)$						
40.0	0.36	MOSEK	42.90	3.77269	40	1.12
		Gurobi	39.27	3.77269	34	1.28
		ADMM	6.50	3.77255	460	0.025
57.6	0.52	MOSEK	43.08	3.77334	46	1.09
		Gurobi	57.85	3.77333	37	1.77
		ADMM	7.08	3.77330	340	0.035
102.4	0.92	MOSEK	116.40	3.77412	43	3.13
		Gurobi	164.12	3.77413	43	4.13
		ADMM	11.66	3.77410	320	0.071
160.0	1.44	MOSEK	303.89	3.77459	50	6.66
		Gurobi	298.47	3.77460	45	7.14
		ADMM	17.27	3.77458	300	0.13

Adaptive Mesh Solution. With the use of AMR, the best LB estimation we may obtain is 3.775063 using linear stress elements, the adaptive refined mesh is shown in Figure 8.

We may observe from Figure 8 that the refined zone matches with the slip-line zone reported in previous work by Martin [33].

5.2.4. Discussion

After examining the three 2D benchmark problems individually, we summarize the overall timing performance of the solvers in Figure 9. The figure

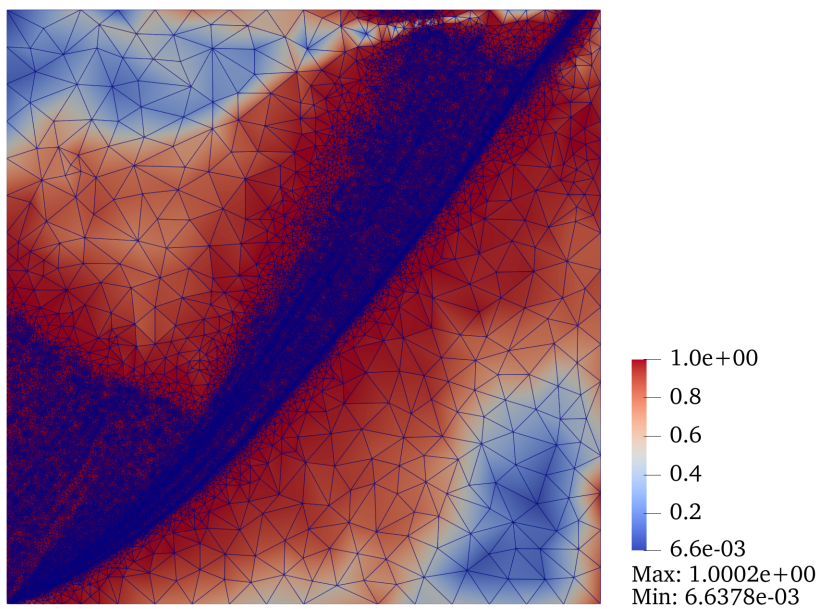


Figure 8: AMR mesh of the vertical cut problem in plane strain

compares the time to solution of ADMM, MOSEK, and Gurobi as the problem size increases for the rigid platen, strip footing, and vertical cut problems. Gurobi is often faster than MOSEK for smaller problem instances, but its performance deteriorates significantly for larger problems. In contrast, the proposed ADMM solver consistently achieves the shortest solution times while maintaining high solution accuracy, with the absolute error in the collapse multiplier remaining below 10^{-4} . This speedup is primarily attributed to the substantially lower cost per ADMM iteration compared with that of the IPM solvers, together with the fact that the number of ADMM iterations increases only moderately as the problem size grows.

The irregular spikes observed in the MOSEK timings for the rigid platen and strip footing problems are primarily attributed to a larger number of interior-point iterations required for those particular cases.

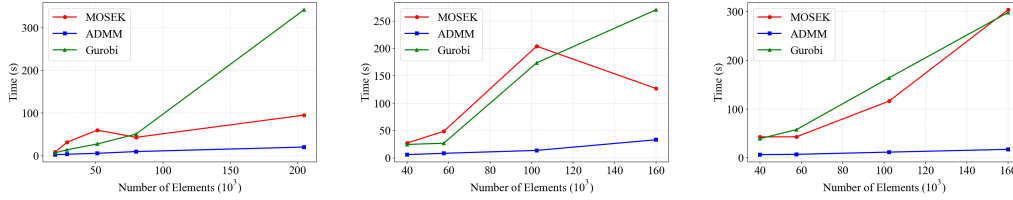


Figure 9: Timing comparison of ADMM against IPM solvers in 2D benchmark problems (From left to right: Rigid platen, strip footing, vertical cut)

5.3. 3D Benchmark Problems

5.3.1. Rigid Platen Problem

Problem setup. This example considers the 3D compression of a von Mises soil block between two rigid platens, as illustrated in Figure 10. The soil domain has dimensions $L \times L \times 0.5L$. The soil is assumed to obey the von-Mises yield criterion. We exploit symmetry on the lower, frontal and diagonal boundaries, while the upper rigid platen is subjected to a vertical live load F_{live} . The soil-platen interfaces are taken to be fully rough; therefore, no constraints need to be placed on the interface stresses. The remaining right boundary is left free. By exploiting symmetry, only one sixteenth of the model needs to be analysed.

Benchmark comparison. Table 4 shows that our solver still yields a speedup, from $2.04\times$ to $5.00\times$ compared with MOSEK in 3D FELA problems. Speedup

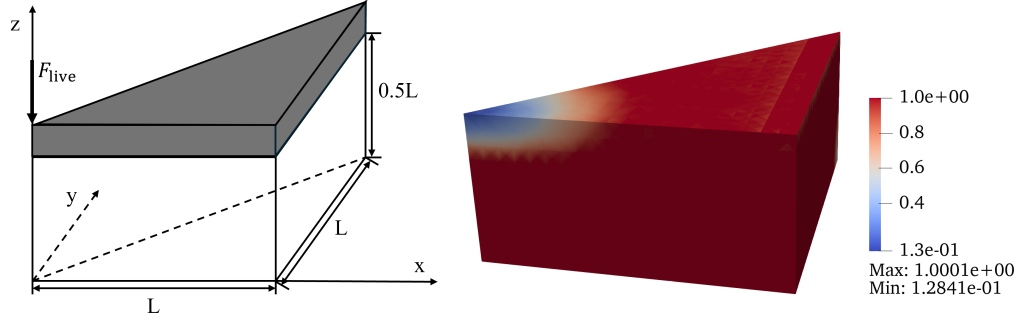


Figure 10: 3D rigid platen problem. **Left:** physical configuration; **Right:** solution of the yield function using a structured mesh

Table 4: Solver performance comparison on the 3D rigid platen benchmark for different mesh discretizations

Problem size		Solver	Solve time (s)	Collapse multiplier	Iterations	Time-per-iter (s)
$N_{\text{elem}}(\times 10^3)N_{\text{var}}(\times 10^6)$						
6	0.148	MOSEK	8.84	2.08904	24	0.37
		Gurobi	10.15	2.08905	17	0.60
		ADMM	4.33	2.08844	220	0.019
24.6	0.589	MOSEK	96.63	2.09546	33	2.93
		Gurobi	147.24	2.09546	16	9.20
		ADMM	22.33	2.09526	220	0.10
96	2.304	MOSEK	371.2	2.09755	39	9.52
		Gurobi	618.31	2.09755	17	36.37
		ADMM	61.99	2.09708	180	0.34

is achieved in both linear and linear elements, with the largest speedup of $5.99\times$ achieved at the largest linear case. It should be noted however, the sparsity pattern of the 3D problem differs from that of the 2D problems, thereby incurring a substantial increase in the number of fill-ins in direct factorization, which limited the maximum problem size we can explore.

5.3.2. Square Footing Problem

Problem setup. Similar to the 2D strip footing problem, the square footing problem considers a rigid footing resting on the surface of a soil body. By exploiting symmetry, only one eighth of the full domain is modelled. Symmetry boundary conditions are imposed on the diagonal plane and the frontal plane. The rigid footing is located at the top-left corner of the computational domain, corresponding to the symmetric portion of the full square footing. The footing side length is set to one fifth of the characteristic length of the soil body, i.e. L for a soil domain of characteristic length $5L$. The bottom boundary and the far-field lateral boundary on the right-hand side are fixed, while the remaining external boundaries are left free unless constrained by symmetry or footing contact conditions.

No analytical lower-bound solution appears to be available for the square or rectangular footing problem. However, da Silva and Salgado [12, 43] investigated this problem with Tresca criterion using FELA and with linear stress elements. Salgado [43] reported a lower-bound solution of 5.523 using a tailored mesh with thin elements near the footing edge, whereas da Silva [12] obtained a lower-bound solution of 5.625 on a very fine structured mesh.

Benchmark comparison. The performance of the proposed 3D ADMM solver is first assessed on structured meshes and compared against MOSEK. The results are summarized in Table 5. Similar to the previous benchmarks, both the time-to-solution and the computed collapse multiplier are reported.

Table 5 shows that across all tested cases, the relative discrepancy between the two solvers remains small, confirming that the proposed ADMM can recover high-quality LB solutions for this 3D benchmark. Refining the linear mesh to 96×10^3 elements increases the collapse multiplier to about 5.36. This trend is consistent with the expected convergence of lower-bound FELA solutions under mesh refinement.

In terms of computational performance, the ADMM solver remains faster than MOSEK, with speedups ranging from approximately $1.27\times$ to $2.61\times$. These speedups are more modest than those observed in the 2D strip foot-

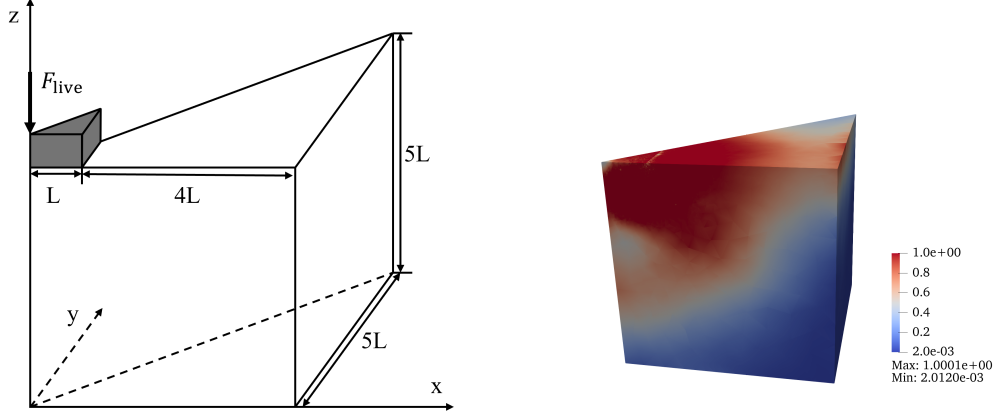


Figure 11: 3D square footing problem. **Left:** physical configuration; **Right:** solution of the yield function using a refined quadratic mesh

Table 5: Solver performance comparison on the 3D square footing benchmark for different mesh discretizations

Problem size		Solver	Solve time (s)	Collapse multiplier	Iterations	Time-per-iter (s)
$N_{\text{elem}}(\times 10^3)N_{\text{var}}(\times 10^6)$						
12	0.288	MOSEK	20.87	5.22710	27	0.77
		Gurobi	31.93	5.22716	16	1.99
		ADMM	17.65	5.22694	420	0.042
96	2.304	MOSEK	937.53	5.36728	32	29.29
		Gurobi [†]	—	—	—	≈ 200
		ADMM	538.21	5.36173	540	0.99

[†] Gurobi did not complete the larger problem. The reported estimated time per iteration was approximately 200s; therefore, the run was terminated due to the excessive computational time.

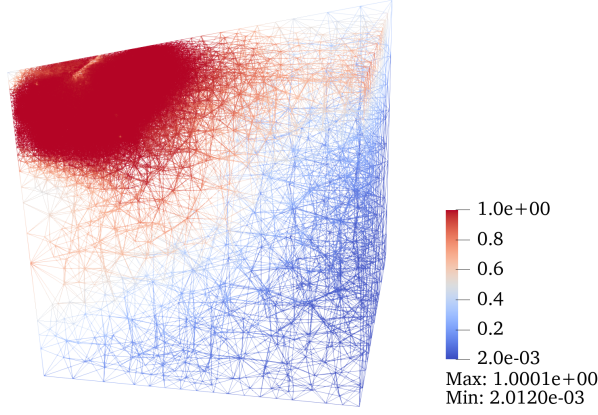


Figure 12: AMR result of the square footing problem

ing benchmark and in the 3D rigid platen problem. This is because the square footing problem produces a more demanding three-dimensional KKT system, for which sparse direct factorization introduces substantially more fill-in. Consequently, the cost of storing and applying the factorized matrices becomes a dominant part of the total solution time. The benchmark therefore highlights both the robustness of the proposed solver and the memory-related scalability bottleneck of the current direct-solve implementation in 3D.

Adaptive mesh refinement. Here we demonstrate the ability of our methodology to obtain an accurate LB solution in 3D. We were able to obtain an accurate LB solution of $\beta = 5.40200$. This solution is obtained with 8 adaptive mesh refinements with 0.18 million linear stress elements, and the adaptive mesh obtained is shown in Figure 12.

The AMR result demonstrates that the proposed framework can be extended to challenging three-dimensional FELA problems and can automatically concentrate resolution in the critical yielding region as shown in the upper left corner of the soil body in Figure 12. Nevertheless, the process was stopped after 8 refinements because the excessive fill-in introduced by sparse direct factorization exhausted the available GPU device memory. This limitation arises from the direct factorization-solve routine, and could potentially be alleviated in larger 3D problems by adopting an indirect, matrix-free

linear solver.

5.3.3. Vertical Cut Problem

Problem Setup. In 3D, the geometry of the cut is characterized by the aspect ratio H/B , where H denotes the depth of the cut and B denotes its width. Unless otherwise stated, we consider the case $H/B = 5$, with $H = 1$, $B = 0.2$, and $L = 2$, so that the computational domain is sufficiently large to contain the potential failure mechanism. Similar to its 2D counterpart, the soil is modelled as a von Mises material subjected to a uniform body force representing the live load. The vertical cut surface and the top surface are taken as free boundaries; the front and back diagonal surfaces are treated as symmetry planes; and the right and bottom surfaces are fixed. The physical configuration is illustrated in Figure 13. Exploiting symmetry, only one quarter of the soil domain is modelled.

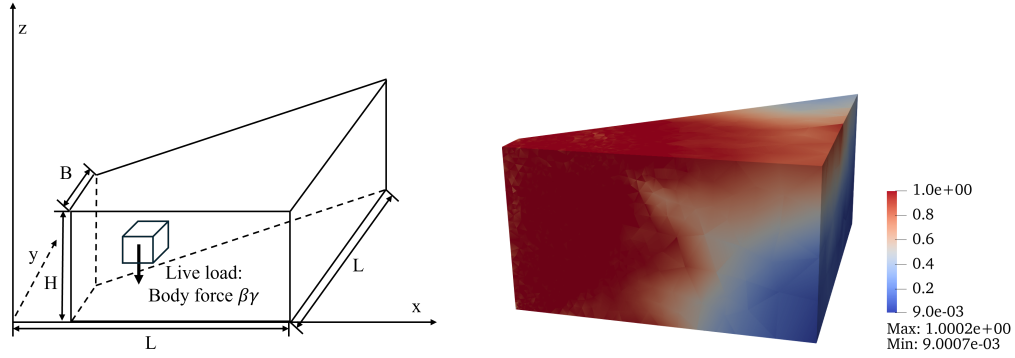


Figure 13: 3D vertical cut problem. **Left:** physical configuration; **Right:** solution of the yield function using a refined mesh

Adaptive Mesh Solution. Adaptive mesh refinement enables the failure mechanism of the vertical cut to be resolved in greater detail. Figure 14 presents the corresponding adaptive mesh in wireframe form. For the formulation employing linear stress elements, the best load multiplier obtained is $\beta = 6.55712$. The failure zone is clearly captured by the highly refined red region adjacent to the vertical excavation.

5.3.4. Discussion

As in the 2D examples, the proposed ADMM solver remains competitive in the 3D benchmarks, achieving a speedup of approximately $2\times$ to $4\times$ over

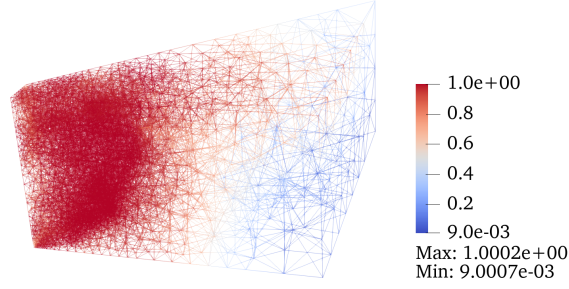


Figure 14: AMR result of the vertical cut problem in 3D

MOSEK. Gurobi is consistently slower across the cases considered. Although the speedup is less pronounced than that observed for the 2D problems, the underlying reasons are discussed in the following section. Nevertheless, the computational cost per ADMM iteration remains substantially lower than that of an IPM iteration. Moreover, with warm-starting and active-cone identification, the number of ADMM iterations varies only moderately as the problem size increases.

The use of adaptive mesh refinement also yields tighter lower-bound solutions and provides a more detailed representation of the failure region. However, a clearly defined 3D 'slip surface' is not fully resolved, primarily because achieving the required level of mesh refinement would incur prohibitive computational cost.

6. Discussion

A vanilla ADMM implementation with GPU acceleration can already be competitive with IPMs on certain large-scale SOCPs. This is because ADMM internally benefits from cheaper iterations since the coefficient matrix remains unchanged throughout the iteration process and therefore is only factorized once, after which the factorization is reused in every subsequent `solve()`.

6.1. Effect of solver enhancements

Beyond this baseline, our work introduces two mechanisms that substantially improve ADMM performance on the large sparse SOCPs arising from lower-bound FELA in particular:

1. **Active-cone identification with adaptive updates.** Since only a subset of cones is typically active at optimality, we periodically detect potentially active conic constraints and update the relevant entries of $D(\rho)$, which tends to improve convergence by emphasizing the active part of the feasible set.
2. **Coarse-to-fine continuation via warm-starting.** We warm-start fine-mesh solves with coarse-mesh solutions, exploiting ADMM’s sensitivity to initialization to reduce the iterations required to meet the same stopping criteria. This idea is not limited to computational mechanics and has the potential to extend to other large-scale optimization settings involving a family of closely related problems.

Both approaches substantially reduced the iteration count required for ADMM to reach the same termination tolerances. Also, with the adaptive updates on the penalty matrix $D(\rho)$, the convergence speed of ADMM now relies less on the initial selection of ρ since the penalty parameter is adjusted dynamically during the iterations. The results also indicate that the required iteration count remains approximately independent of the problem size.

Here, we discuss the specific improvement made by each individual upgrade to the overall solver performance in the 3D rigid platen problem. The subdivision used is a structured mesh, with 20 subdivisions on the longest side and therefore has 48000 linear tetrahedral stress elements, with 1.152 million variables, as shown in the last entry of Table 4. A specific performance comparison of using different solver options is given below, and here we choose the solver using the augmented matrix (AUG for short) with warm-starting (WS) and cone identification (CI) as the baseline.

Table 6: Solver performance with different solver options

Solver options	Solve time (s)	Iterations	Analysis time (s)	Factorization time (s)	β
AUG + Projection	41.30	140	19.20	18.31	2.09477
AUG + WS + CI	61.99	180	19.58	16.21	2.09708
AUG + CI	110.74	280	19.82	16.01	2.09229
AUG + WS	98.34	600	19.96	16.09	2.09750
Condensed	352.05	440	37.35	45.00	2.09749

As shown in Table 6, the use of the three-dimensional ellipsoidal projection leads to the best overall performance among the tested configurations. Compared with the baseline AUG + WS + CI solver, AUG + Projection reduces the number of ADMM iterations from 180 to 140 and decreases the total solve time from 61.99s to 41.30s, while producing a comparable collapse multiplier. This indicates that the projection-based treatment improves the convergence behaviour of the solver for this problem. Moreover, the projection operation is naturally amenable to GPU parallelization and introduces only a small overhead, accounting for approximately 3% of the total runtime.

The condensed formulation, by contrast, incurred the highest computational cost. As shown in Table 6, its analysis and factorization times were substantially greater than those of the augmented formulation. This is primarily due to the increased fill-in produced during the direct factorization of the condensed system. Consequently, the memory required to store the factors exceeded the available GPU memory, introducing additional GPU I/O overhead and further degrading the overall performance.

The effects of warm-starting and active cone identification can also be observed from the comparison between AUG + WS, AUG + CI, and AUG + WS + CI. Using active cone identification alone reduces the number of iterations compared with warm-starting alone, from 600 to 280. However, it results in a longer solve time because identifying the active cones changes the matrix entries and therefore requires repeated matrix updates and re-factorizations. Warm-starting alone avoids this additional factorization cost, but requires significantly more ADMM iterations. Combining warm-starting with active cone identification provides the best balance between iteration reduction and

factorization overhead between these two enhancements, reducing the solve time to 61.99s.

6.2. Scalability bottleneck of direct factorization in 3D

The availability of high-performance GPUs makes it possible to solve 2D and 3D FELA problems with adaptive mesh refinement and higher-order elements, often obtaining accurate lower-bound solutions within a few minutes. The direct solve routine is computationally efficient as long as the factorization fits within GPU device memory and does not require out-of-core memory I/O operations. Nevertheless, the proposed GPU-based direct factorization approach still faces a fundamental scalability bottleneck: its dependence on GPU device memory. As the problem size increases, especially in 3D simulations, the KKT system grows rapidly and the memory required to store the sparse factors can become prohibitive. This may ultimately limit the largest problem size that can be solved on a given GPU.

This limitation was not observed even in the largest 2D benchmarks. However, it became significant for large-scale 3D discretizations, even when the total number of variables was similar. The main reason is that the cost of sparse direct factorization is governed not only by the dimension of the linear system, but also by the sparsity pattern and the amount of fill-in generated during factorization. In 3D finite element problems, the factorization process typically leads to much denser matrices and substantially more fill-in than in 2D problems. As a result, both the numerical factorization and the repeated triangular solves required in the ADMM iterations can become dominated by memory traffic and I/O operations.

This difference is particularly pronounced in FELA. In LB formulations, each stress point involves more stress components in 3D problems than in 2D, leading to larger local blocks and bandwidth in the assembled system. The sparsity pattern of the A matrix in FELA is demonstrated in Figure 15.

Here, the 3D finite element connectivity introduces stronger and more spatially extensive coupling between variables in the inter-element equilibrium part of the constraint matrix A . Apart from that, the extra dimension increases the number of equality constraints required per stress point from 2 to 3 in intra-element equilibrium, and the dimension of the SOC constraints increased from 3×3 to 6×6 in 3D, when using Drucker-Prager or von Mises yield criterion.

Consequently, even for problems with a similar number of variables, the resulting KKT systems may exhibit much denser factors after reordering and

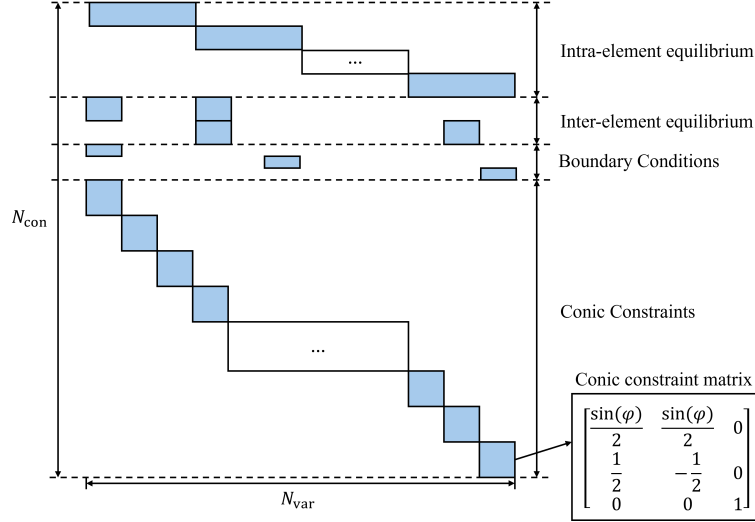


Figure 15: Sparsity pattern of the constraint matrix A

factorization. This explains why the direct GPU-based approach remains highly efficient for the 2D plane strain cases studied here, but becomes more challenging for large-scale 3D discretizations.

6.3. Potential of indirect and matrix-free ADMM solvers

Nevertheless, a more scalable direction may be to avoid explicitly computing and storing the sparse factorization altogether, and instead adopt an indirect linear solution strategy within ADMM. This is especially relevant for three-dimensional FELA, where direct factorization of sparse matrices can quickly become unfavourable due to excessive fill-in [44]. As noted in the broader numerical linear algebra literature, iterative solvers are often preferable, and in many large-scale three-dimensional applications practically necessary, due to the memory and computational costs associated with sparse direct methods [8]. Several ADMM-based solvers already provide indirect linear-system solvers [20, 17, 19, 9]. Notably, SCS has reported speedups from its GPU-based indirect implementation on large-scale problems [20], suggesting that the combination of ADMM and indirect solves can be practical and scalable for sufficiently large instances.

6.4. Generality, limitations, and future extensions

The proposed method achieves a speedup over the IPM on many large-scale problems arising in FELA, while maintaining accuracy comparable to

MOSEK under a similar accuracy target. We note, however, that part of this speedup is achieved using a GPU-based implementation, whereas MOSEK is run on a multi-threaded CPU.

While the proposed acceleration techniques are developed in the context of LB-FELA, they target generic bottlenecks of first-order methods for SOCPs. Therefore, they may also benefit other sparse SOCPs with PDE-like structures. A systematic evaluation on broader classes of problems is left for future work.

Apart from the advances made in the GPU-accelerated ADMM solver, this study also implemented a simple but effective adaptive mesh refinement strategy that relies only on the lower-bound solution. The proposed framework was able to obtain a tight and strict lower-bound solution within one minute in 2D examples. This improvement stems not only from the acceleration provided by the GPU solver, but also from the computational efficiency of the higher-order Bernstein formulation [24].

Future work will also consider more advanced yield criteria, such as Mohr-Coulomb and Lade-Duncan. Incorporating these criteria would increase the practical value of the proposed methodology, as they are widely used in geotechnical and engineering applications. Within the ADMM framework, such criteria can be incorporated provided that the projection of a general stress state onto the admissible yield surface can be computed efficiently [12, 14]. For example, although not necessarily in the context of FELA, Clausen and co-workers implemented the three-dimensional Mohr-Coulomb criterion using a return-mapping strategy, in which the stress state is transformed into principal stress space, projected onto the yield surface, and then transformed back to the original stress space [45]. Extending this type of projection strategy to the present FELA-ADMM framework is a promising direction for future research.

7. Conclusion

This paper presents a scalable GPU-accelerated ADMM framework for lower-bound finite element limit analysis. By leveraging ADMM together with tailored acceleration strategies, the proposed method provides an efficient solution pipeline for large-scale FELA problems. The framework is validated systematically on 2D benchmark problems and further extended to representative 3D applications.

Across the considered benchmarks, the proposed solver delivers substantial reductions in time-to-solution relative to a state-of-the-art interior-point solver, while maintaining essentially identical LB collapse multipliers. In particular, the numerical results demonstrate that the speedups are achieved without sacrificing solution quality, supporting the use of the proposed approach as a practical alternative for large-scale conic optimization problems. Combined with adaptive mesh refinement, the solver is able to obtain a tight, strict LB solution of a large-scale 2D problem on a modest GPU. Representative 3D benchmarks show that the proposed solver remains computationally effective beyond the 2D setting, while large-scale adaptive mesh refinement simulations further highlight its scalability and practical applicability. Together, these results indicate that the proposed method combines strong performance in 2D with promising extensibility to 3D lower-bound FELA.

In addition, the integration of adaptive mesh refinement (AMR) and Bernstein elements further improves the quality of the computed strict lower bound, enabling high-quality solutions to be obtained efficiently for challenging geometries and complex geotechnical stress fields. The projection-based formulation also highlights the flexibility of the proposed ADMM framework, indicating its potential to accommodate more sophisticated yield criteria within the same computational pipeline.

Overall, the combination of scalable conic optimization, GPU acceleration, mesh adaptivity, and higher-order elements provides an effective workflow toward high-fidelity limit analysis at problem sizes that are difficult to handle with conventional second-order methods.

References

- [1] L. Vandenberghe, The cvxopt linear and quadratic cone program solvers, Online: <http://cvxopt.org/documentation/coneprog.pdf> 53 (2010).
- [2] A. Domahidi, E. Chu, S. Boyd, Ecos: An socp solver for embedded systems, in: 2013 European control conference (ECC), IEEE, pp. 3071–3076.
- [3] E. D. Andersen, K. D. Andersen, The MOSEK interior point optimizer for linear programming: an implementation of the homogeneous algorithm, Springer, 2000, pp. 197–232. doi:10.1007/978-1-4757-3216-0_8. URL https://link.springer.com/chapter/10.1007/978-1-4757-3216-0_8
- [4] P. J. Goulart, Y. Chen, Clarabel: An interior-point solver for conic programs with quadratic objectives, arXiv preprint arXiv:2405.12762 (2024).
- [5] N. Podlich, A. V. Lyamin, S. W. Sloan, High performance interior point methods for three-dimensional finite element limit analysis, in: COMPLAS XV: proceedings of the XV International Conference on Computational Plasticity: fundamentals and applications, CIMNE, pp. 121–132.
- [6] N. C. Podlich, The development of efficient algorithms for large-scale finite element limit analysis, Thesis (2017).
- [7] J. Gondzio, Interior point methods 25 years later, European journal of operational research 218 (3) (2012) 587–601. doi:10.1016/j.ejor.2011.09.017.
- [8] Y. Saad, Iterative methods for sparse linear systems, 2nd Edition, Iterative methods for sparse linear systems, Society for Industrial and Applied Mathematics SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104, Philadelphia, Pa, 2003, includes bibliographical references p. 495-516) and index.
- [9] T. Diamandis, Z. Frangella, S. Zhao, B. Stellato, M. Udell, Genios: an (almost) second-order operator-splitting solver for large-scale convex optimization, Mathematical programming computation (2026). doi:10.1007/s12532-025-00301-2.

- [10] J. Gondzio, Matrix-free interior point method, *Computational optimization and applications* 51 (2) (2012) 457–480. doi:10.1007/s10589-010-9361-3.
- [11] F. Zanetti, J. Gondzio, A new stopping criterion for krylov solvers applied in interior point methods, *SIAM journal on scientific computing* 45 (2) (2023) A703–A728. doi:10.1137/22M1490041.
- [12] M. V. da Silva, N. Deusdado, A. N. Antao, Lower and upper bound limit analysis via the alternating direction method of multipliers, *Computers and Geotechnics* 124 (2020). doi:10.1016/j.compgeo.2020.103571.
URL <https://www.sciencedirect.com/science/article/abs/pii/S0266352X20301348>
- [13] A. N. Antao, M. V. da Silva, N. Monteiro, N. Deusdado, Upper and lower bounds for three-dimensional undrained stability of shallow tunnels, *Transportation Geotechnics* 27 (2021). doi:10.1016/j.trgeo.2020.100491.
URL <https://www.sciencedirect.com/science/article/abs/pii/S2214391220303792>
- [14] A. N. Antão, M. Vicente da Silva, Three-dimensional limit analysis with lade–duncan criterion (2022). doi:10.1680/jgele.22.00015.
- [15] M. Vicente da Silva, A. N. Antão, Computational upper- and lower-bound 3d limit analysis using the hoek–brown yield criterion, *International journal of geomechanics* 24 (4) (2024). doi:10.1061/IJGNALGMENG-9081.
- [16] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, Distributed optimization and statistical learning via the alternating direction method of multipliers, *Foundations and Trends in Machine Learning* 3 (2011) 1–122. doi:10.1561/22000000016.
- [17] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, S. Boyd, Osqp: An operator splitting solver for quadratic programs, *Mathematical Programming Computation* 12 (4) (2020) 637–672.
- [18] P. C. N. Verheijen, D. Goswami, M. Lazar, Superadmm: Solving quadratic programs faster with dynamic weighting admm, in: *International Conference on System Theory, Control and Computing*, IEEE, pp. 569–575. doi:10.1109/ICSTCC66753.2025.11240524.

- [19] M. Garstka, M. Cannon, P. Goulart, Cosmo: A conic operator splitting method for convex conic problems, *Journal of Optimization Theory and Applications* 190 (3) (2021) 779–810. doi:10.1007/s10957-021-01896-x. URL <https://link.springer.com/article/10.1007/s10957-021-01896-x>
- [20] B. O’Donoghue, E. Chu, N. Parikh, S. Boyd, Conic optimization via operator splitting and homogeneous self-dual embedding, *Journal of optimization theory and applications* 169 (3) (2016) 1042–1068. doi:10.1007/s10957-016-0892-3.
- [21] N. Rontsis, P. Goulart, Y. Nakatsukasa, Efficient semidefinite programming with approximate admm, *Journal of Optimization Theory and Applications* (2022) 1–29.
- [22] N. Corporation, Nvidia cudss (preview): A high-performance cuda library for direct sparse solvers (2025). URL <https://docs.nvidia.com/cuda/archive/12.9.0/cudss/index.html>
- [23] A. Makrodimopoulos, C. M. Martin, Lower bound limit analysis of cohesive-frictional materials using second-order cone programming, *International Journal for Numerical Methods in Engineering* 66 (4) (2006) 604–634. doi:10.1002/nme.1567. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/nme.1567>
- [24] C. Martin, Finite element limit analysis using higher-order element, in: L. Zdravkovic, S. Kontoe, D. M. G. Taborda, A. Tsiamposi (Eds.), *10th European Conference on Numerical Methods in Geotechnical Engineering*, p. 16. doi:<https://doi.org/10.53243/NUMGE2023-439>.
- [25] A. Makrodimopoulos, A fundamental class of stress elements in lower bound limit analysis, *Proceedings of the Royal Society A* 476 (2243) (2020) 20200425.
- [26] A. Makrodimopoulos, A class of strain-displacement elements in upper bound limit analysis, *International Journal for Numerical Methods in Engineering* 123 (16) (2022) 3681–3712.

- [27] H. S. Yu, S. Sloan, P. W. Kleeman, Quadratic element for upper bound limit analysis, *Engineering Computations* 11 (1994) 195–212. doi:10.1108/02644409410799281.
- [28] A. Makrodimopoulos, C. M. Martin, Upper bound limit analysis using discontinuous quadratic displacement fields, *Communications in Numerical Methods in Engineering* 24 (11) (2008) 911–927. doi:10.1002/nag.567.
- [29] E. Christiansen, O. S. Edmund, Automatic mesh refinement in limit analysis, *International Journal for Numerical Methods in Engineering* 50 (2001) 1331–1346. doi:10.1002/1097-0207(20010228)50:6<1331::AID-NME46>3.0.CO;2-S.
- [30] A. V. Lyamin, S. W. Sloan, K. Krabbenhøft, M. Hjiaj, Lower bound limit analysis with adaptive remeshing, *International Journal for Numerical Methods in Engineering* 63 (14) (2005) 1961–1974. doi:<https://doi.org/10.1002/nme.1352>.
URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/nme.1352>
- [31] H. Ciria, J. Peraire, J. Bonet, Mesh adaptive computation of upper and lower bounds in limit analysis, *International journal for numerical methods in engineering* 75 (8) (2008) 899–944.
- [32] R. Sun, K. Zhang, H. Cai, Three-dimensional analysis of shield tunnel face stability using adaptive lower bound finite element limit analysis with the drucker–prager criterion, *Geotechnical and geological engineering* 44 (1) (2026) 43. doi:10.1007/s10706-025-03527-y.
- [33] C. Martin, The use of adaptive finite-element limit analysis to reveal slip-line fields, *GEOTECHNIQUE LETTERS* 1 (2) (2011) 23–29.
- [34] J. R. Shewchuk, Triangle: Engineering a 2d quality mesh generator and delaunay triangulator, *Applied Computational Geometry Towards Geometric Engineering*, Springer Berlin Heidelberg, pp. 203–222.
- [35] H. Si, Tetgen, a delaunay-based quality tetrahedral mesh generator, *ACM transactions on mathematical software* 41 (2) (2015) 1–36. doi:10.1145/2629697.

- [36] M. Schubiger, G. Banjac, J. Lygeros, Gpu acceleration of admm for large-scale quadratic programming, *Journal of Parallel and Distributed Computing* 144 (2020) 55–67. doi:10.1016/j.jpdc.2020.05.021.
URL <https://www.sciencedirect.com/science/article/pii/S0743731520303063>
- [37] J. Salençon, Théorie des charges limites: poinçonnement d’une plaque par deux poinçons symétriques en déformation plane, *Comptes Rendus Mécanique*, Acad. Sc. Paris 265 (1967) 869–872.
- [38] W. F. Hosford, R. M. Caddell (Eds.), *Slip-Line Field Analysis*, 3rd Edition, Cambridge University Press, Cambridge, 2007, pp. 128–162. doi:DOI: 10.1017/CBO9780511811111.010.
URL <https://www.cambridge.org/core/product/E1154A885F3D381E4AB27573532247C8>
- [39] S. W. Sloan, Geotechnical stability analysis, *Geotechnique* 63 (7) (2013) 531–572. doi:10.1680/geot.12.RL.001.
URL <https://www.emerald.com/jgeot/article-abstract/63/7/531/402093/Geotechnical-stability-analysis?redirectedFrom=fulltext>
- [40] L. Prandtl, Hauptaufsätze: Über die eindringungsfestigkeit (härte) plastischer baustoffe und die festigkeit von schneiden, *Zeitschrift für angewandte Mathematik und Mechanik* 1 (1) (1921) 15–20. doi:10.1002/zamm.19210010102.
- [41] C. Martin, Exact bearing capacity calculations using the method of characteristics, *Proc. 11th Int. Conf. of IACMAG Turin* 4 (2005).
URL <https://api.semanticscholar.org/CorpusID:118276564>
- [42] D. Kong, C. M. Martin, B. W. Byrne, Modelling large plastic deformations of cohesive soils using sequential limit analysis, *International Journal for Numerical and Analytical Methods in Geomechanics* 41 (18) (2017) 1781–1806. doi:10.1002/nag.2700.
URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/nag.2700>
- [43] R. Salgado, A. V. Lyamin, S. W. Sloan, H. S. Yu, Two- and three-dimensional bearing capacity of foundations in clay, *Géotechnique* 54 (5) (2004) 297–306. doi:10.1680/geot.2004.54.5.297.

- [44] J. R. Shewchuk, An introduction to the conjugate gradient method without the agonizing pain.
- [45] J. Clausen, L. Damkilde, L. Andersen, An efficient return algorithm for non-associated plasticity with linear yield criteria in principal stress space, *Computers & structures* 85 (23) (2007) 1795–1807. doi:10.1016/j.compstruc.2007.04.002.